

Práctica: Docker y Docker Compose

Sistemas de Gestión Empresarial · Ejercicios

7.1. Imágenes de Docker

1. Descarga la versión 15.0 de la imagen de Odoo.

```
~/Proyectos/DAM2/Sistema Gestión Empresarial/Tema2/Actividades/docker master*
> docker pull odoo:15.0
15.0: Pulling from library/odoo
d29fc5639ff0: Pull complete
15ff680154e9: Pull complete
60c9c6706b01: Pull complete
fa0650a893c2: Pull complete
da66d955686e: Pull complete
d909df565121: Pull complete
01ebfbd0c10fc: Pull complete
4c45db42d990: Pull complete
e906b2624dd1: Pull complete
9262ffa545ad: Download complete
767324ffe333: Download complete
Digest: sha256:9843efa8a0c8fba20f81a97fd04339fe0cc49c3162485c3e8bc0447295434ac1
Status: Downloaded newer image for odoo:15.0
docker.io/library/odoo:15.0
```

2. Comprueba el espacio ocupado en disco por cada una de las imágenes descargadas.

```
~/Proyectos/DAM2/Sistema Gestión Empresarial/Tema2/Actividades/docker master*
> docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
bytemark/webdav:latest	21eed0df1238	142MB	32.2MB	U
odoo:15.0	9843efa8a0c8	2.3GB	599MB	

3. ¿Qué diferencia existe entre ejecutar `docker pull odoo` y `docker pull odoo:16.0`?

Docker pull odoo me descargará la ultima versión disponible.

7.2. Creación y gestión de contenedores

4. Detén el contenedor `mi_postgres` y vuélvelo a iniciar.

```
~/Proyectos/DAM2/Sistema Gestión Empresarial/Tema2/Actividades/docker master*
> docker stop postgresDocker
postgresDocker

~/Proyectos/DAM2/Sistema Gestión Empresarial/Tema2/Actividades/docker master*
> docker start postgresDocker
postgresDocker
```

5. Consulta los logs del contenedor `mi_postgres` para comprobar que se inició correctamente.

```
PostgreSQL Database directory appears to contain a database; Skipping initialization
2026-09-30 16:07:10.600 UTC [1] LOG: starting PostgreSQL 15.19 (Debian 15.19-1.pgdg13+2) on x86_64-pc-linux-gnu, compiled by gcc (Debian 14.2.0-19) 14.2.0, 64-bit
2026-09-30 16:07:10.600 UTC [1] LOG: listening on IPv4 address "0.0.0.0", port 5432
2026-09-30 16:07:10.600 UTC [1] LOG: listening on IPv6 address ":::", port 5432
2026-09-30 16:07:10.602 UTC [1] LOG: listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
2026-09-30 16:07:10.605 UTC [29] LOG: database system was shut down at 2026-09-30 16:06:38 UTC
2026-09-30 16:07:10.608 UTC [1] LOG: database system is ready to accept connections
```

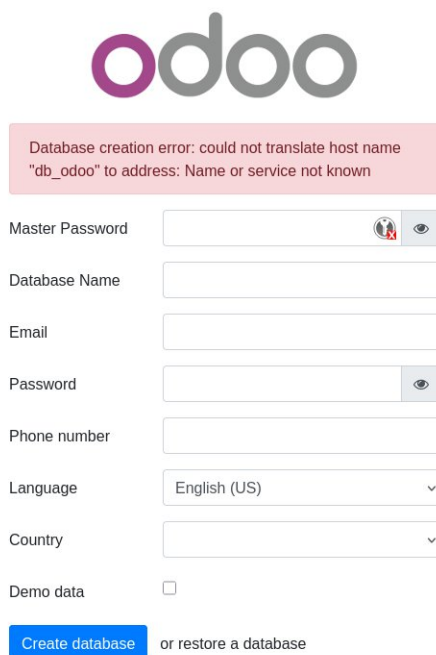
6. Entra por la consola interactiva (bash) al contenedor `mi_postgres` y ejecuta el comando `psql -U odoo -d postgres` para verificar la base de datos.

```
~/Proyectos/DAM2/Sistema Gestión Empresarial/Tema2/Actividades/docker master*
> docker exec -it postgresDocker bash
root@794b62127586:/# psql -U odoo -d postgres
psql (15.19 (Debian 15.19-1.pgdg13+2))
Type "help" for help.
```

7. Ejecuta un contenedor temporal de Odoo mapeando el puerto `8069` del contenedor al puerto `8069` de tu máquina local.

```
/datos/alexont/docker
> docker run -d \
  --name app_odoo \
  --network red_odoo \
  -p 8069:8069 \
  -v odoo_web_data:/var/lib/odoo \
  -e HOST=db_odoo \
  -e USER=odoo \
  -e PASSWORD=odoo_pass \
  odoo:15.0
0f5cb9de998936627b961130303ee3a77a0deb4c4aa1feb6b11182661c34626c
```

8. Accede a `http://localhost:8069` en tu navegador y comprueba qué sucede al no estar conectado a PostgreSQL.



The screenshot shows the Odoo logo at the top. Below it is a red error message box that reads: "Database creation error: could not translate host name 'db_odoo' to address: Name or service not known". Underneath the error message is a form with the following fields: "Master Password" (with a toggle for visibility), "Database Name", "Email", "Password" (with a toggle for visibility), "Phone number", "Language" (set to "English (US)"), "Country", and "Demo data" (with an unchecked checkbox). At the bottom of the form is a blue button labeled "Create database" followed by the text "or restore a database".

9. Detén y elimina todos los contenedores creados en esta sección.

```
/datos/alexont/docker
> docker stop app_odoo
app_odoo

/datos/alexont/docker
> docker rm app_odoo
app_odoo
```

7.3. Gestión de volúmenes y persistencia de datos

10. Crea un contenedor de PostgreSQL utilizando el volumen `db_data_odoo` montado en la ruta interna `/var/lib/postgresql/data`.

```
/datos/alexont/docker
> docker run -d \
  --name postgres_persistente \
  -v db_data_odoo:/var/lib/postgresql/data \
  -e POSTGRES_USER=odoo \
  -e POSTGRES_PASSWORD=odoo_clave \
  postgres:15
7f717e7f3c3f014e52d67526730dfceb09fa36c2a4c2ce062bbef32da2c6ec16
```

```
/datos/alexont/docker
> docker run -d \
  --name postgres_persistente \
  -v db_data_odoo:/var/lib/postgresql/data \
  -e POSTGRES_USER=odoo \
  -e POSTGRES_PASSWORD=odoo_clave \
  postgres:15
7f717e7f3c3f014e52d67526730dfceb09fa36c2a4c2ce062bbef32da2c6ec16
```

11. Averigua la ruta física del sistema anfitrión donde Docker almacena los volúmenes usando `docker volume inspect`.

```
/datos/alexont/docker
> docker volume inspect db_data_odoo
[
  {
    "CreatedAt": "2026-09-30T20:57:17+02:00",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/db_data_odoo/_data",
    "Name": "db_data_odoo",
    "Options": null,
    "Scope": "local"
  }
]
```

12. Detén el contenedor, elimínalo y posteriormente borra el volumen `db_data_odoo` con `docker volume rm`.

```
/datos/alexont/docker
> docker stop postgres_persistente
postgres_persistente

/datos/alexont/docker
> docker rm postgres_persistente
postgres_persistente

/datos/alexont/docker
> docker volume rm db_data_odoo
db_data_odoo
```

7.4. Redes en Docker

13. Inspecciona la red mediante `docker network inspect red_odoo` para comprobar las direcciones IP asignadas.

```
/datos/alexont/docker
> docker network inspect red_odoo
[
  {
    "Name": "red_odoo",
    "Id": "705571b036f4b7d08569f442b85ec692f6f383ca3ac7974cf482a698c86bcb7b",
    "Created": "2026-09-30T20:57:44.635166227+02:00",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv4": true,
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": {},
      "Config": [
        {
          "Subnet": "172.19.0.0/16",
          "Gateway": "172.19.0.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Options": {},
    "Labels": {},
    "Containers": {},
    "Status": {
      "IPAM": {
        "Subnets": {
          "172.19.0.0/16": {
            "IPsInUse": 3,
            "DynamicIPsAvailable": 65533
          }
        }
      }
    }
  }
]
```

14. Entra por consola interactiva al contenedor de Odoo e comprueba la conectividad hacia el contenedor de base de datos haciendo `ping db_odoo`.

```
odoo@38b1bc8e38d7:/$ ping db_odoo
bash: ping: command not found
odoo@38b1bc8e38d7:/$
```

15. Detén y elimina ambos contenedores, y elimina la red creada usando `docker network rm red_odoo`.

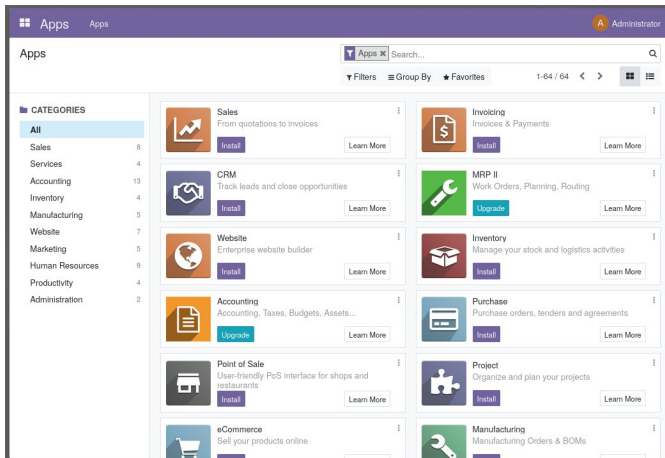
```
/datos/alexont/docker
> docker stop app_odoo && docker rm app_odoo
app_odoo
app_odoo

/datos/alexont/docker
> docker stop db_odoo && docker rm db_odoo
db_odoo
db_odoo

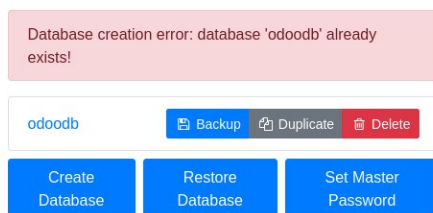
/datos/alexont/docker
> docker network rm red_odoo
red_odoo
```

7.5. Orquestación completa con docker compose

- 16. Despliegue:** Guarda el archivo `docker-compose.yml`, levanta el entorno y crea una base de datos de prueba desde el navegador web.



- 17. Comprobación de Persistencia:** Ejecuta `docker compose down`, vuelve a levantar los servicios con `docker compose up -d` y verifica que la información creada en Odoo siga intacta.



- 18. Personalización:** Modifica el archivo `docker-compose.yml` para descargar odoo en su version 19 y vincula una carpeta local llamada `./addons` con la ruta interna de Odoo `/mnt/extra-addons`.

```
GNU nano 9.2 docker-compose.yml
services:
  web:
    image: odoo:19.0
    container_name: odoo_web
    depends_on:
      - db
    ports:
      - "8069:8069"
    environment:
      - HOST=db
      - USER=odoo
      - PASSWORD=odoo_secret_password
    volumes:
      - odoo-web-data:/datos/alexont/Proyectos/DAM2/Sistema Gestión Empresarial/Tema2/Actividades/docker/odoo
      - ./addons:/mnt/extra-addons
  db:
    image: postgres:15
    container_name: odoo_db
    environment:
      - POSTGRES_DB=postgres
      - POSTGRES_USER=odoo
      - POSTGRES_PASSWORD=odoo_secret_password
    volumes:
      - odoo-db-data:/datos/alexont/Proyectos/DAM2/Sistema Gestión Empresarial/Tema2/Actividades/docker/data
volumes:
  odoo-web-data:
  odoo-db-data:
```

19. Capturas de pantalla a incluir en la entrega:

- Salida de los comandos `docker images` y `docker ps`.

```
/datos/alexont/Proyectos/DAM2/Sistema Gestión Empresarial/Tema2/Actividades/docker master*
> docker images

IMAGE                                ID                                DISK USAGE  CONTENT SIZE  EXTRA
bytemark/webdav:latest               21eed0df1238                      142MB        32.2MB        U
odoo:15.0                             9843efa8a0c8                      2.3GB        599MB         U
odoo:19.0                             77bac5cd1e06                      3.23GB       753MB         U
postgres:15                          724292da1f2e                      630MB        164MB         U

/datos/alexont/Proyectos/DAM2/Sistema Gestión Empresarial/Tema2/Actividades/docker master*
> docker ps

CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                                NAMES
b488dc71b8f3   odoo:19.0                  "/entrypoint.sh odoo"   5 minutes ago Up 5 minutes   0.0.0.0:8069->8069/tcp, [::]:8069->8069/tcp, 8071-8072/tcp   odoo_web
86fe9e13c44a   postgres:15                "docker-entrypoint.s..." 5 minutes ago Up 5 minutes   5432/tcp                                           odoo_db
e89c1090/a5f    bytemark/webdav            "docker-entrypoint.s..." 5 weeks ago   Up 25 hours   443/tcp, 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   keepass_webdav

/datos/alexont/Proyectos/DAM2/Sistema Gestión Empresarial/Tema2/Actividades/docker master*
> █
```

- Archivo `docker-compose.yml` modificado con el volumen local de addons.

```
/datos/alexont/Proyectos/DAM2/Sistema Gestión Empresarial/Tema2/Actividades/docker master*
> cat docker-compose.yml
services:
  web:
    image: odoo:19.0
    container_name: odoo_web
    depends_on:
      - db
    ports:
      - "8069:8069"
    environment:
      - HOST=db
      - USER=odoo
      - PASSWORD=odoo_secret_password
    volumes:
      - odoo-web-data:/datos/alexont/Proyectos/DAM2/Sistema Gestión Empresarial/Tema2/Actividades/docker/odoo
      - ./addons:/mnt/extra-addons
  db:
    image: postgres:15
    container_name: odoo_db
    environment:
      - POSTGRES_DB=postgres
      - POSTGRES_USER=odoo
      - POSTGRES_PASSWORD=odoo_secret_password
    volumes:
      - odoo-db-data:/datos/alexont/Proyectos/DAM2/Sistema Gestión Empresarial/Tema2/Actividades/docker/data
volumes:
  odoo-web-data:
  odoo-db-data:
```

- Interfaz web de Odoo funcionando correctamente.

